

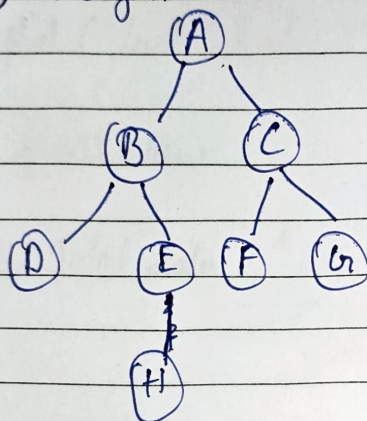
LAB 2a: Breadth-First Search

Given to find a path from A to H from,
 $A \rightarrow B, C$ $B \rightarrow D, E$ $C \rightarrow F, G$ $E \rightarrow H$
 (D, F, G, H are leaves)

To code $\rightarrow$ A to H path and to return its traversal order

To answer $\rightarrow$ which line defines BFS or DFS and why.

Given, We get:-



Since we are gonna do BFS we need search by level.

Code:

graph = { 'A': ['B', 'C'], 'B': ['D', 'E'],
 'C': ['F', 'G'], 'D': [], 'E': ['H'],
 'F': [], 'G': [], 'H': [] }

def BFS (start, goal)

~~queue~~

queue = [start]

v = [] #visited


```

while queue:
    path = queue.pop(0)
    node = path[-1]
    if node == goal:
        return path, v
    if node not in V:
        v.append(node)
        for n in graph[node]:
            queue.append(path + [n])

```

return None, v

```

p, vi = BFS('A', 'H')
print("BFS path :", p)
print("BFS visited :", vi)

```

Output:

BFS path : ['A', 'B', 'E', 'H']
 BFS visited : ['A', 'B', 'C', 'D', 'E', 'F', 'G']

Answer:

As we know BFS follows FIFO while DFS follows LIFO, the line:

→ path = queue.pop() (which requires reversed pop)
 → This decides the search strategy as DFS it would have been:

path = stack.pop()

With BFS it finds by each level so, it would go A, B, E, H directly while in DFS it will track back.